

Lưu ý dành cho các giáo viên và người dùng khác của những slide này: Chúng tôi rất vui nếu bạn thấy tài liệu của chúng tôi hữu ích cho việc giảng dạy của mình. Bạn có thể sử dụng các slide này nguyên văn hoặc chỉnh sửa chúng cho phù hợp với nhu cầu của riêng mình. Nếu bạn sử dụng một phần đáng kể các slide này trong bài giảng của mình, vui lòng ghi kèm thông báo này hoặc liên kết đến trang web của chúng tôi: <http://www.mmm>

CS246 2025/26: Mining Massive Data Sets Intro, MapReduce & Spark

CS246: Khai thác các tập dữ liệu khổng lồ

Jure Leskovec, Đại học Stanford

Charilaos Kanatsoulis

<http://cs26.stanford.edu>





Dữ liệu chứa đựng giá trị và kiến thức.

Data Mining, Data Science, ...

Nhưng để trích xuất dữ liệu kiến thức
cần phải

Lưu trữ (hệ thống)

Quản lý (cơ sở dữ liệu)

Và PHÂN TÍCH lớp học này

Khai thác dữ liệu $\approx$ Phân tích dự đoán $\approx$

Khoa học dữ liệu $\approx$ Học máy $\approx$

Trí tuệ nhân tạo tập trung vào dữ liệu

What This Course Is About

Trích xuất thông tin hữu ích từ

(Thường) là các tập dữ liệu rất lớn, là chủ đề gây ra sự cường điệu, lo ngại và quan tâm cực độ.

Không phải tất cả đều xoay quanh máy học

Nhưng phần lớn là như vậy!

Trọng tâm của CS246 là các thuật toán có khả năng mở rộng.

Song song hóa thường rất cần thiết

Data Mining/ML Pipeline

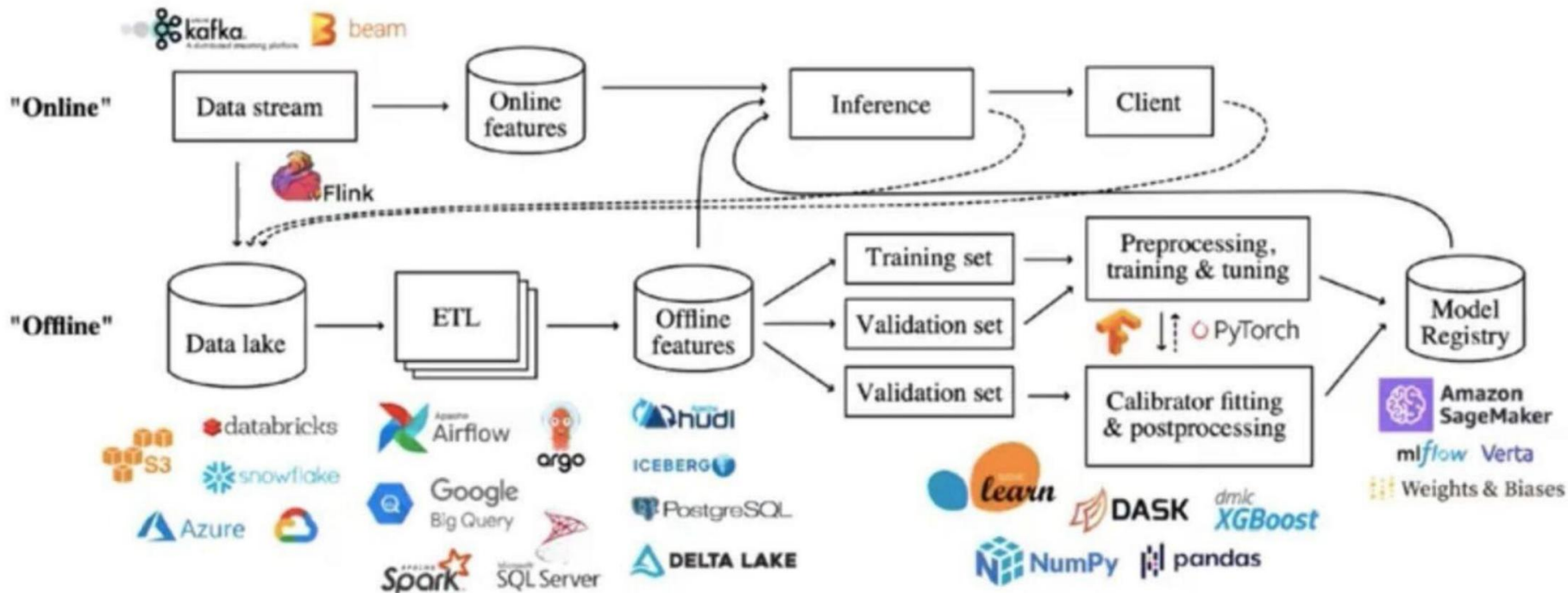


Figure 1: High-level architecture of a generic end-to-end machine learning pipeline. Logos represent a sample of tools used to construct components of the pipeline, illustrating heterogeneity in the tool stack. Shankar et al. 2021

“Môn học này là bắt buộc nếu bạn muốn trở thành nhà khoa học dữ liệu hoặc kỹ sư học máy.” (một sinh viên giấu tên của khóa học CS246)

Data Mining Methods

Phương pháp mô tả

Tìm các mẫu hình mà con người có thể hiểu được để mô tả dữ liệu

Ví dụ: Phân cụm

Phương pháp dự đoán

Sử dụng một số biến để dự đoán các giá trị chưa biết hoặc giá trị tương lai của các biến khác.

Ví dụ: Hệ thống đề xuất

“Chắc chắn nên tham gia khóa học này nếu bạn dự định làm việc với các tập dữ liệu khổng lồ trong tương lai, dù là trong ngành công nghiệp hay trong giới học thuật.” (một sinh viên giấu tên của khóa học CS246)

This Class: CS246

Điều này kết hợp những ưu điểm tốt nhất của máy học,
thống kê, trí tuệ nhân tạo, cơ sở dữ liệu nhưng cần nhấn mạnh
hơn vào...

Khả năng mở rộng (dữ liệu lớn)

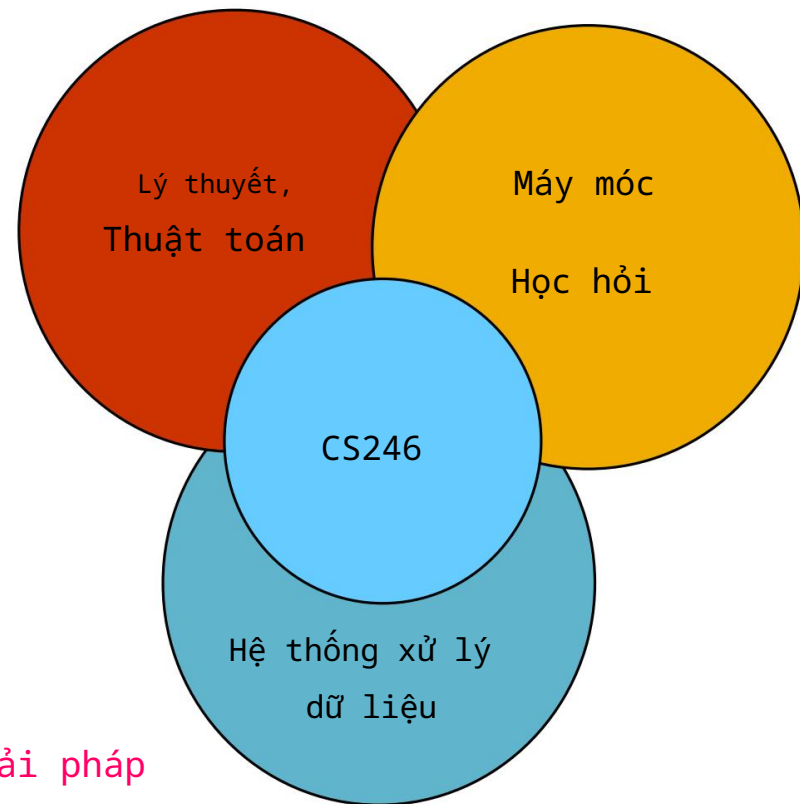
Thuật toán

Kiến trúc máy tính

Tự động hóa để xử lý dữ
liệu lớn

“Lớp học tập trung rất nhiều vào các trường hợp
nghiên cứu thực tế , vì vậy bạn sẽ học được rất
nhiều về các vấn đề học máy thực tiễn và các giải pháp
đang được sử dụng trong thực tế tại các công ty như

Netflix, Amazon, Facebook, Pinterest, v.v.” (một sinh viên CS246 giấu tên)



What will we learn?

Chúng ta sẽ học cách khai thác các loại dữ liệu khác nhau:

- Dữ liệu có nhiều chiều

- Dữ liệu là một đồ thị

- Dữ liệu là vô hạn/không bao giờ kết thúc

- Dữ liệu được gắn nhãn

Chúng ta sẽ học cách sử dụng các mô hình khác nhau của
tính toán:

- MapReduce

- Luồng phát và thuật toán trực tuyến

- Bộ nhớ trong của một máy duy nhất

What will we learn?

Chúng ta sẽ học cách **giải quyết các vấn đề thực tế:**

Hệ thống đề xuất Phân

tích giỏ hàng Phát hiện

thư rác Lọc dữ

liệu Chúng ta

sẽ tìm hiểu **nhiều “công cụ” khác nhau:**

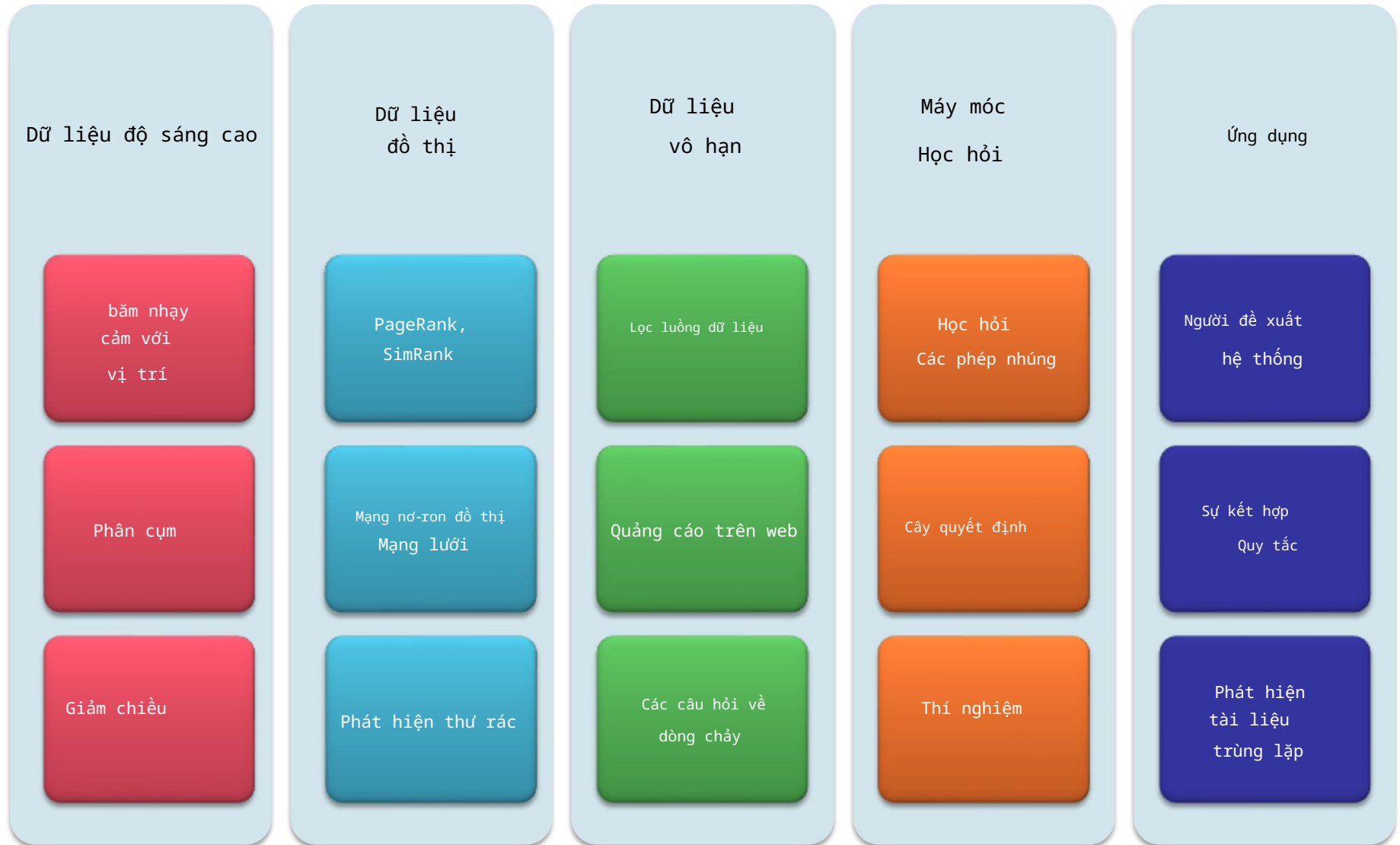
Đại số tuyến tính (SVD, Hệ thống hồi quy, Cộng

đồng) Tối ưu hóa (phương pháp giảm độ dốc

ngẫu nhiên) Lập trình động (tập hợp mục

thường xuyên) Băm (LSH, bộ lọc Bloom)

How the Class Fits Together



Course Logistics

Course Staff

Instructor



Jure Leskovec

Course Assistants



Hongyue Li (Head TA)



Tim Chen



Harper Hua

Guest Co-Instructor



Charilaos Kanatsoulis



Ayush Agrawal



Josh Sanyal

Course Format: Zoom&Canvas

Thời gian giảng bài: Thứ Ba/Thứ Năm, 15:00-16:20 (giờ Thái Bình Dương)

Buổi học trực tiếp (tại phòng học NVIDIA), bản ghi sẽ có trên Canvas.

Bài giảng khoảng 70 phút:

Nếu bạn có thắc mắc cần làm rõ, hãy giơ tay

Hỏi đáp (~10 phút):

Hãy đặt câu hỏi, chúng tôi sẽ trả lời và thảo luận.

Logistics: Communication

Biên tập viên:

Sử dụng Ed cho tất cả các câu hỏi và giao tiếp công cộng.

Hãy tìm kiếm trong nguồn cấp dữ liệu trước khi đặt câu hỏi trùng lặp.

Vui lòng gắn thẻ bài đăng của bạn và không đăng bài quá ngắn gọn (chỉ có một dòng).

Khi gửi email cho giảng viên, hãy luôn sử dụng:

cs246-win2526-staff@lists.stanford.edu

Chúng tôi sẽ đăng thông báo khóa học lên Ed (vì vậy hãy kiểm tra thường xuyên nhé!)

Khách dự thính cũng được chào

đón! (vui lòng gửi yêu cầu đến [<cs246-win2526-staff@lists.stanford.edu>](mailto:cs246-win2526-staff@lists.stanford.edu)) (để thêm bạn vào Canvas)

Logistics: Communication

Phản hồi tần số cao:

Khảo sát hàng tuần về tinh thần lớp học

Chọn ngẫu nhiên một số sinh viên để nhận phản hồi từ họ.

Nội dung

Thiết lập khóa học

Những điều mà đội ngũ giảng dạy cần biết/cần cải thiện

Bất cứ điều gì khiến bạn bối rối

-

Resources

Trang web khóa học: <http://cs246.stanford.edu>

Bài giảng (cần nộp ít nhất 30 phút trước giờ giảng)

Bài tập về nhà, lời giải, tài liệu tham khảo được đăng tải trên Ed/Canvas

Sách giáo khoa: Khai thác dữ liệu khổng lồ (Mining of Massive Datasets) của A. Rajaraman, J. Ullman và J. Leskovec

Được bán bởi Nhà xuất bản Đại học Cambridge nhưng có thể tải xuống miễn phí tại <http://mmds.org>

Khóa học trực tuyến (MOOC): www.youtube.com/channel/UC_0ao2FYkLAU1UVkBfze4jg/videos

CS246 Office Hours

Giờ làm việc:

Giờ làm việc của trợ giảng sẽ được cập nhật trên trang web <http://cs246.stanford.edu> trước thứ Sáu

Chúng ta sẽ bắt đầu giờ tư vấn trực tuyến vào tuần tới!

Giờ làm việc sẽ được tổ chức trên Zoom và sử dụng

Trạng thái hàng đợi

Các liên kết sẽ được đăng trên Canvas và lịch học của khóa học.

Chúng tôi sẽ tổ chức (1) giờ làm việc trực tiếp, (2) giờ làm việc trực tuyến và có thể (3) kết hợp trực tuyến + trực tiếp giờ làm việc

Recitation Sessions

Video và tài liệu trên Canvas

Hướng dẫn sử dụng Spark:

Chi tiết sẽ được thông báo sau.

Theo dõi Colab 0

Ôn tập các kiến thức cơ bản về xác suất và kỹ thuật chứng minh:

Thông tin chi tiết sẽ được thông báo sau, tài liệu [phát tay](#).

Ôn tập đại số tuyến tính:

Thông tin chi tiết sẽ được thông báo sau, tài liệu [phát tay](#).

Work for the Course: Homework

4 bài tập về nhà dài hơn: 40%

Bốn bài tập chính, bao gồm lập trình, chứng minh và phát triển thuật toán.

Bài tập tốn rất nhiều thời gian (hơn 20 giờ). Hãy bắt đầu sớm!!

Cách thức nộp bài?

Bài tập về nhà cần viết báo cáo:

Nộp bài qua [Gradescope](#)

Đăng ký môn CS246 trên Canvas, và bạn sẽ tự động được chấp nhận.
đã được thêm vào Gradescope của khóa học

Mã bài tập về nhà:

Nếu bài tập về nhà yêu cầu nộp mã lập trình, bạn sẽ tìm thấy một bài tập riêng biệt trên Gradescope, ví dụ: HW1 (Code)

Quên nộp mã sẽ bị trừ điểm.

Homework Calendar

Lịch trình bài tập về nhà:

Ngày (23:59 PT) Ra ngoài		TRONG
01/08, Thứ Năm	Bài tập về nhà 1	
22/01, Thứ Năm	Bài tập về nhà 2	Bài tập về nhà 1
02/05, Thứ Năm	Bài tập về nhà 3	Bài tập về nhà 2
19/02, Thứ Năm	Bài tập về nhà 4	Bài tập về nhà 3
03/05, Thứ Năm		Bài tập về nhà 4

Hai đợt nộp bài tập về nhà muộn cho quý này:

Thời hạn nộp muộn sẽ hết hạn vào thứ Hai tuần sau, lúc 23:59 giờ PST.

Có thể sử dụng tối đa 1 khoảng thời gian trễ cho mỗi bài tập về nhà

Work for the Course: Colabs

Sổ tay Colab ngắn hàng tuần: 30%

Các bài đăng trên Colab được đăng tải vào mỗi thứ Năm.

Tổng cộng có 10 phần, từ 0 đến 9, mỗi phần trị giá 3%

Hạn chót là một tuần sau, vào thứ Năm lúc 23:59 giờ PST. **Không chấp nhận trễ hạn!**

Hai bài đăng hợp tác đầu tiên sẽ được đăng tải vào thứ Năm, kèm theo thông tin chi tiết.

hướng dẫn nộp bài lên Gradescope

Bài tập Colab 0 (Hướng dẫn Spark) được giải từng bước trong video hướng dẫn _____

[Spark](#) .

Việc cộng tác cần khoảng 1 giờ làm việc.

Và một vài dòng mã.

“Colab” là một dịch vụ điện toán đám mây miễn phí từ Google, dùng để lưu trữ Jupyter.

máy tính xách tay có quyền truy cập GPU và TPU miễn phí

Work for the Course: Final Exam

Bài thi cuối kỳ: 30%

Kế hoạch dự kiến: Thi **trực tiếp** vào tuần cuối kỳ

Thứ Năm, ngày 20 tháng 3 năm 2025, từ 12:15 đến 15:15.

Điểm thưởng: Tỷ lệ thuận với đóng góp của bạn (tối đa 2%)

Tham gia khóa học, đặt câu hỏi, thảo luận

Vì đã tham gia các cuộc thảo luận về giáo dục

Đặc biệt có giá trị là câu trả lời cho các câu hỏi do đặt ra.
các học sinh khác

Báo cáo lỗi trong tài liệu khóa học

Prerequisites

Lập trình: Python hoặc Java

Các thuật toán cơ bản: Môn CS161 chắc chắn là đủ.

Xác suất: ví dụ như CS109 hoặc Stats116

Sẽ có một buổi ôn tập và tài liệu ôn tập được đính kèm trên trang chủ của lớp học.

Đại số tuyến tính:

Có sẵn thêm tài liệu ôn tập và buổi ôn tập khác.

Giải tích đa biến

Hệ thống cơ sở dữ liệu (SQL, đại số quan hệ):

CS145 là đủ nhưng không bắt buộc

What If I Don't Know All This Stuff?

Mỗi chủ đề được liệt kê đều quan trọng đối với một phần của khóa học:

Nếu bạn thiếu một hoặc hai kiến thức nền, bạn có thể cân nhắc việc học bổ sung kịp thời những kiến thức cần thiết.

Ngoại lệ là lập trình:

Để học tốt khóa học này, bạn thực sự cần phải Thành thạo lập trình bằng Python hoặc Java.

Honor Code – (1)

Chúng ta sẽ tuân theo quy trình tiêu chuẩn của Khoa Khoa học Máy tính: Bạn có thể nhận trợ giúp, nhưng bạn **PHẢI** ghi nhận sự trợ giúp đó trong bài tập bạn nộp.

Việc không ghi rõ nguồn tham khảo là **vi phạm Quy tắc ứng xử**.

Chúng tôi sử dụng MOSS để kiểm tra tính xác thực của sản phẩm của bạn.
mã số

Honor Code – (2)

Bạn có thể thảo luận với người khác về thuật toán (hoặc các thuật toán) cần sử dụng để giải bài tập về nhà;

Miễn là bạn đề cập đến tên của họ trong bài làm bạn nộp.

Bạn không nên sử dụng mã nguồn của người khác hoặc xem mã nguồn của người khác khi viết mã của riêng mình:

(Không tìm kiếm/đăng tải mã nguồn trên Github, Co-pilots và các trang tương tự)

Bạn có thể trao đổi với mọi người nhưng phải tự viết giải pháp/mã của riêng mình.

Nếu bạn không nêu rõ nguồn tham khảo, MOSS sẽ phát hiện ra, dẫn đến vi phạm quy định về bảo mật thông tin.

Final Thoughts

Môn CS246 có nhịp độ nhanh!

Yêu cầu trình độ lập trình thành thạo

Kỹ năng toán học tốt

Sinh viên SCPD thường quên mất kiến thức toán/lý thuyết.

Thời gian cần thiết cho khóa học: “Các bài tập nhóm khá dễ và có thể hoàn thành trong vòng một giờ, nhưng bài tập về nhà tốn nhiều thời gian hơn nên hãy bắt đầu sớm nhé!” (Sinh viên CS246)

Bài tập về nhà mất khoảng 20 giờ

Việc tạo notebook trên Colab mất khoảng 1 giờ.

Thành lập các nhóm học tập!

Final Thoughts

CS246 là một trong những môn học hữu ích nhất mà bạn sẽ học tại Stanford nếu muốn trở thành nhà khoa học dữ liệu hoặc kỹ sư học máy.

CS246 sẽ vừa thú vị vừa vất vả. _____

Summary of Next Steps

Xem video hướng dẫn trên Colab0

- o Liên kết cũng sẽ được đăng trên Ed.
- o Buổi tư vấn trực tuyến (OH) dành cho các câu hỏi về Spark sẽ diễn ra vào cuối tuần.

Giờ làm việc bắt đầu từ tuần thứ 2.

Lịch trình sẽ được đăng tải trên trang web.

Các sản phẩm sắp ra mắt:

HW1, Colab0 và Colab 1 sẽ được phát hành vào ngày...

Thứ năm

Các bài nộp sắp tới:

- o Colab0, Colab 1 hạn nộp vào ngày 15 tháng 1.

Bài tập về nhà 1 hạn nộp ngày 22 tháng 1.

Distributed Computing for Data Mining

Large-scale Computing

Tính toán quy mô lớn cho các bài toán khai thác dữ liệu trên phần cứng thông thường

Thách thức:

Làm thế nào để phân bổ khả năng tính toán?

Làm thế nào để chúng ta có thể viết các chương trình phân tán một cách dễ dàng hơn?

Máy móc bị hỏng:

Một máy chủ có thể hoạt động liên tục trong 3 năm (1.000 ngày)

Nếu bạn có 1.000 máy chủ, hãy chuẩn bị tinh thần mất 1 máy chủ mỗi ngày.

Trong số 1 triệu máy móc, mỗi ngày có 1.000 máy bị hỏng!

An Idea and a Solution

Vấn đề:

Việc sao chép dữ liệu qua mạng mất thời gian
tưởng:

Đưa khả năng tính toán đến

dữ liệu Lưu trữ tập tin nhiều lần để đảm

bảo độ tin cậy Spark/Hadoop giải quyết những vấn

đề này Cơ sở hạ tầng lưu trữ - Hệ thống tập tin

Google: GFS. Hadoop: HDFS

Mô hình lập trình

MapReduce

Spark

Storage Infrastructure

Vấn đề:

Nếu các nút bị lỗi, làm thế nào để lưu trữ dữ liệu một cách bền vững? Câu trả lời:

Hệ thống tập tin phân tán

Cung cấp không gian tên tệp toàn cục

Mô hình sử dụng điển hình:

Các tập tin dung lượng lớn (hàng trăm GB đến TB)

Dữ liệu hiếm khi được cập nhật tại chỗ

Việc đọc và thêm dữ liệu là phổ biến.

Distributed File System

Máy chủ khối

Tệp được chia thành các phần liên kề

Thông thường mỗi khối dữ liệu có kích thước từ 16-64MB.

Mỗi đoạn dữ liệu được sao chép (thường là 2 hoặc 3 lần)

Cố gắng để các bản sao ở các giá khác nhau

Nút chính

Hay còn gọi là Name Node trong HDFS của Hadoop

Lưu trữ siêu dữ liệu về vị trí lưu trữ các tệp.

Các nút chính thường có khả năng chống chịu tốt hơn với lỗi phần cứng và vận hành các dịch vụ cụm quan trọng.

Thư viện máy khách để truy cập tệp

Trao đổi với người quản lý để tìm máy chủ khối

Kết nối trực tiếp với máy chủ phân vùng dữ liệu để truy cập dữ liệu.

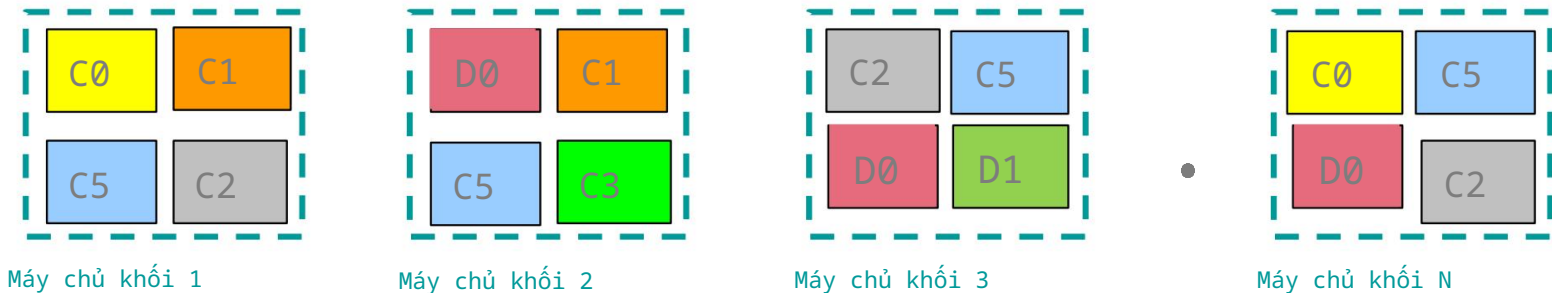
Distributed File System

Hệ thống tập tin phân tán đáng tin cậy

Dữ liệu được lưu trữ theo từng "khối" rải rác trên nhiều máy tính.

Mỗi khối dữ liệu được sao chép trên các máy khác nhau

Khôi phục liên mạch khi xảy ra lỗi ổ đĩa hoặc máy chủ



Ký hiệu: C2. khối số 2 của tập tin C

Đưa quá trình tính toán trực tiếp vào dữ liệu!

Máý chủ khối cũng đóng vai trò là máý chủ tính toán.

MapReduce: Early Distributed Computing Programming Model

Programming Model: MapReduce

MapReduce là một **kiểu lập trình**.

Được thiết kế cho:

1. Lập trình song song dễ dàng
2. Quản lý phần cứng và phần mềm một cách vô hình thất bại
3. Dễ dàng quản lý dữ liệu quy mô rất lớn

Nó có nhiều **cách triển khai khác nhau**, bao gồm:
Hadoop, Spark (được sử dụng trong lớp học này),
Flink và phiên bản gốc của Google chỉ được gọi là...
“MapReduce”

MapReduce: Overview

3 bước của MapReduce

Bản đồ:

Áp dụng **hàm Map** do người dùng viết cho từng phần tử đầu vào

Mapper áp dụng hàm Map cho một phần tử duy nhất

Nhiều bộ xử lý ánh xạ được nhóm lại trong một **tác vụ Ánh xạ** (đơn vị song song hóa)

Kết quả trả về của hàm Map là một tập hợp gồm 0, 1 hoặc nhiều **cặp khóa-giá trị**.

Phân nhóm theo khóa: Sắp xếp và xáo trộn

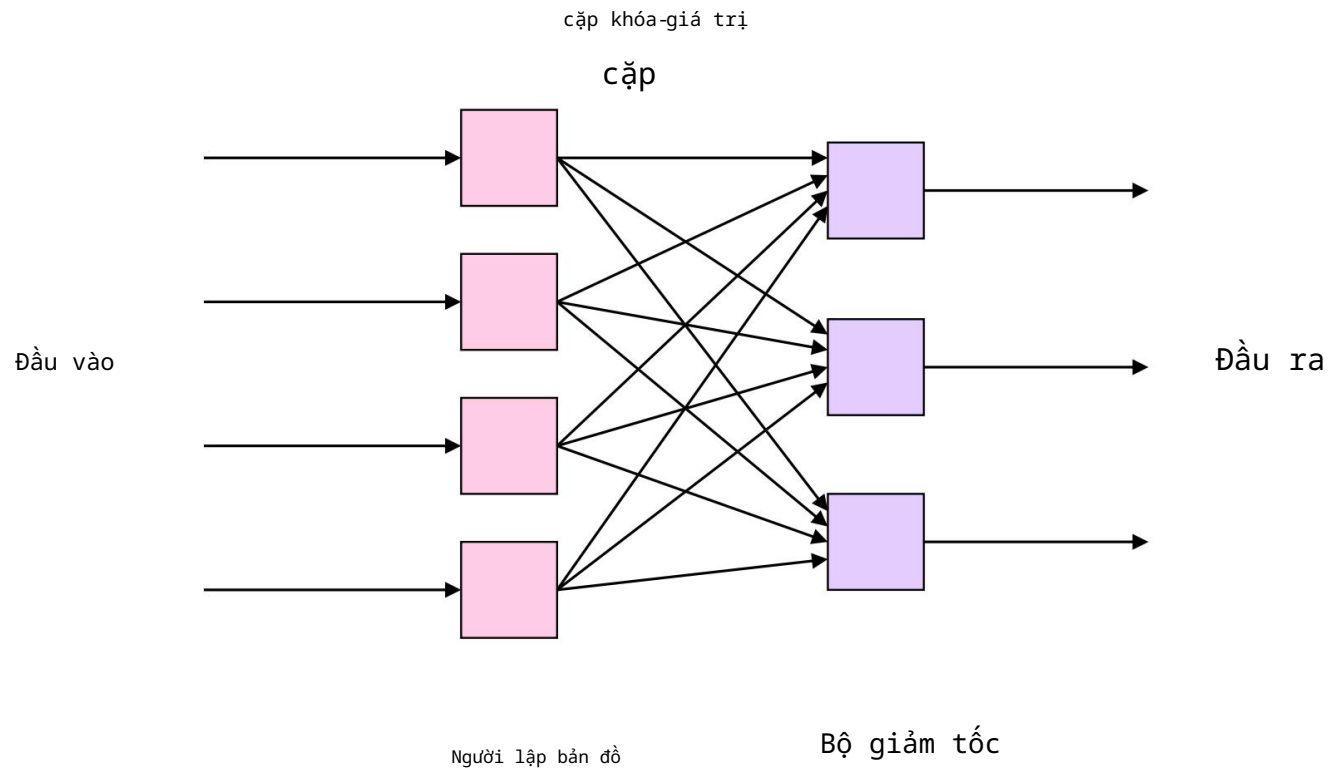
Hệ thống sắp xếp tất cả các cặp khóa-giá trị theo khóa, và xuất ra các cặp khóa-(danh sách giá trị)

Giảm thiểu:

Hàm Reduce do người dùng viết được áp dụng cho từng cặp khóa (danh sách các giá trị).

Sơ đồ bố cục giữ nguyên, còn phần lập bản đồ và thu gọn sẽ thay đổi cho phù hợp với vấn đề.

MapReduce Pattern



Example: Word Counting

Ví dụ về tác vụ MapReduce:

Chúng tôi có một tài liệu văn bản rất lớn.

Đếm số lần xuất hiện của mỗi từ khác

nhau trong tập tin.

Nhiều ứng dụng của điều này:

Phân tích nhật ký máy chủ web để tìm các URL phổ biến

Dịch máy thống kê:

Cần đếm số lần xuất hiện của mỗi chuỗi 5 từ trong một kho tài liệu lớn.

MapReduce: Word Counting

Do lập trình viên
cung cấp.

BẢN ĐỒ:

Đọc dữ liệu đầu vào
và tạo ra một tập hợp
các cặp khóa-giá trị.

Nhóm theo khóa:

Thu thập tất cả các
cặp có cùng khóa

Do lập trình viên
cung cấp.

Giảm bớt:

Thu thập tất cả các giá trị
thuộc về khóa và đầu
ra

Dữ liệu có thể
được phân vùng và xử
lý song song.

Phi hành đoàn của không gian
tàu con thoi Endeavor gần đây
trở về Trái Đất như
những sứ giả, những người báo tin của
một kỷ nguyên mới của không gian
Khám phá. Các nhà khoa học tại
NASA cho biết rằng
cuộc họp gần đây của
Robot Dextre là bước đầu tiên trong
một dự án dài hạn dựa trên không gian
Sự hợp tác giữa con người và máy.
"Công việc chúng ta đang làm hiện nay
-- lĩnh vực robot mà chúng tôi đang nghiên cứu -
- đó là điều chúng ta sẽ làm
nhu cầu

(Cái, 1)

(phi hành đoàn, 1)

(của, 1)

(cái, 1)

(khoảng trắng, 1)

(xe đưa đón, 1)

(Nỗ lực, 1)

(gần đây, 1)

..

(phi hành đoàn, 1)

(phi hành đoàn, 1)

(khoảng trắng, 1)

(cái, 1)

(cái, 1)

(cái, 1)

(xe đưa đón, 1)

(gần đây, 1)

.

(phi hành đoàn, 2)

(khoảng trắng, 1)

(cái, 3)

(xe đưa đón, 1)

(gần đây, 1)

.

Tài liệu lớn

(khóa, giá trị)

(khóa, giá trị)

(khóa, giá trị)

Đầu ra

Word Count Using MapReduce

map(key, value): # key: tên

tài liệu; value: nội dung văn bản của tài liệu

đối với mỗi từ w trong giá trị:

phát ra(w, 1)

reduce(key, values): # key: một từ;

value: một iterator trên các số đếm result = 0

đối với mỗi giá trị đếm v trong:

kết quả $\pm v$

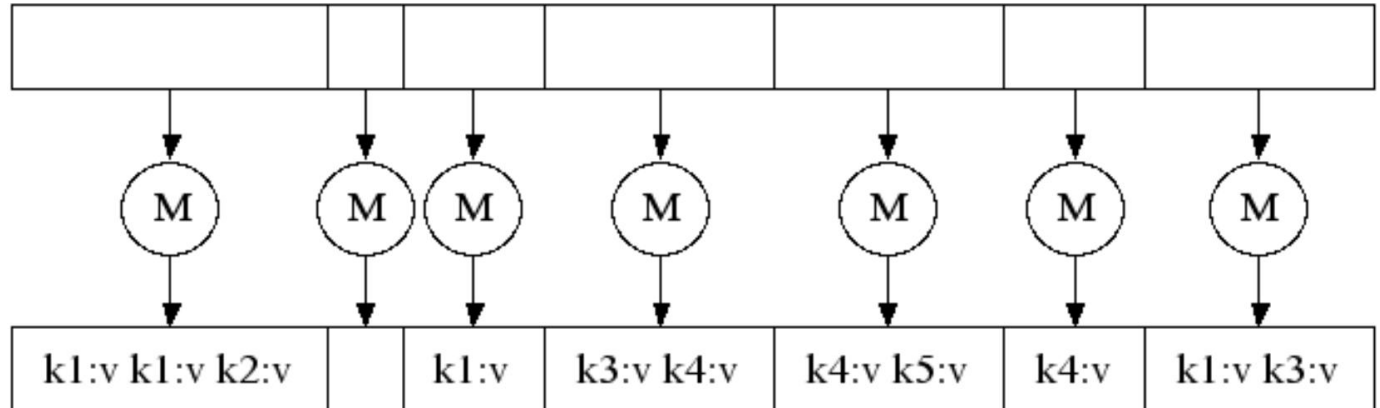
phát ra(khóa, kết quả)

Map-Reduce: In Parallel

BẢN ĐỒ:

Đọc dữ liệu đầu vào
và tạo ra một tập hợp
các cặp khóa-giá trị.

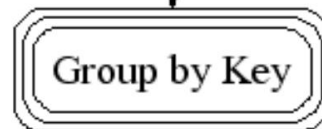
Input



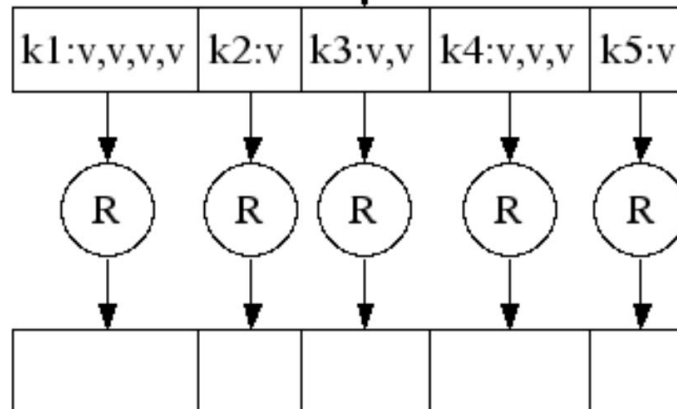
Intermediate

Nhóm theo khóa:

Thu thập tất cả các cặp
có cùng khóa
(Hợp nhất băm, Xáo trộn,
Sắp xếp, Phân vùng)



Grouped

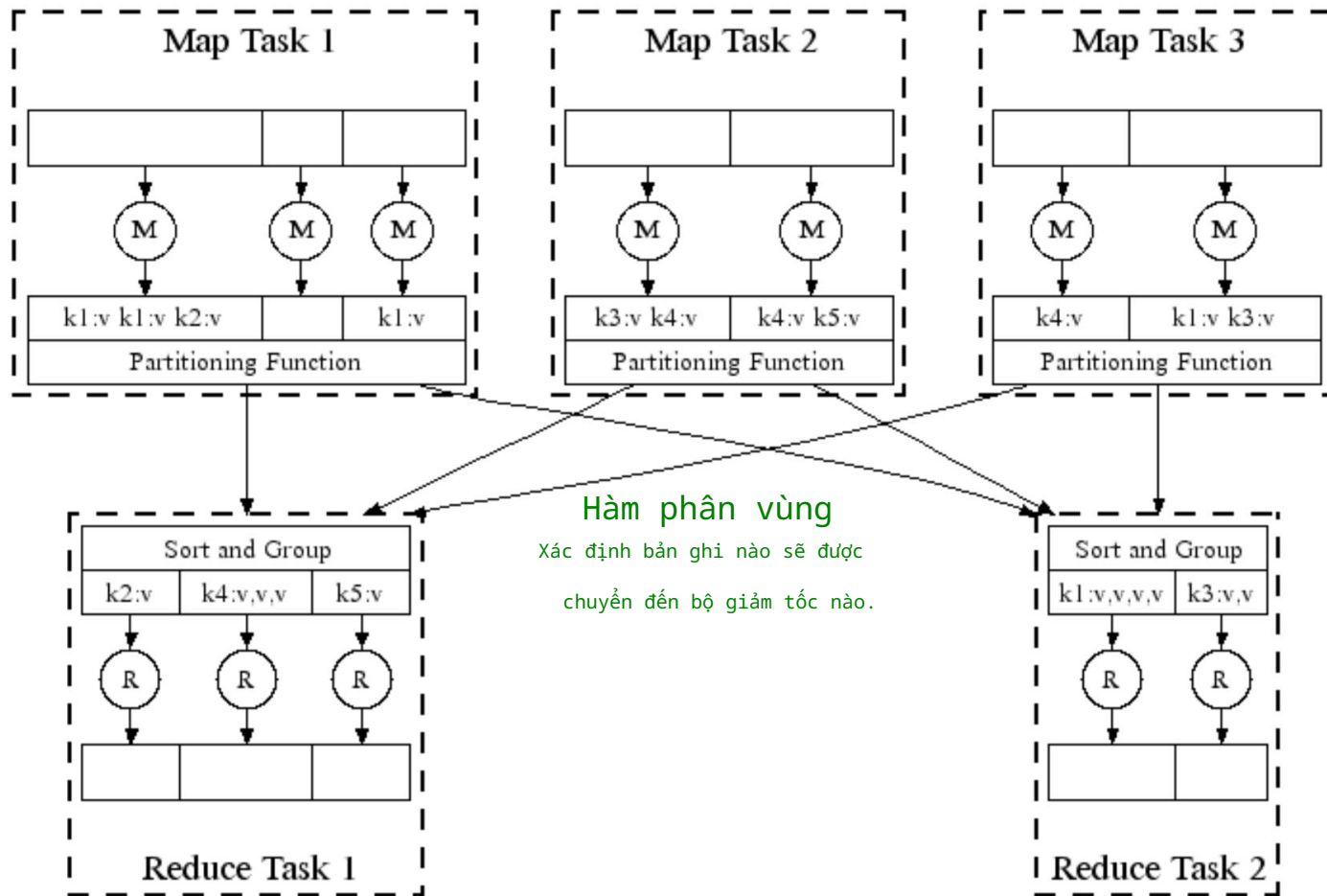


Output

Giảm bớt:

Thu thập tất cả các
giá trị thuộc về
khóa và xuất ra.

Map-Reduce: In Parallel



Các giai đoạn của Map-Reduced được phân tán với nhiều tác vụ thực hiện công việc song song.

MapReduce: Environment

Môi trường MapReduce đảm nhiệm các công việc sau:

Phân vùng dữ liệu đầu vào

Lập lịch thực thi chương trình trên một bộ máy móc

Thực hiện các bước chính theo nhóm

Trên thực tế, đây chính là điểm nghẽn.

Xử lý sự cố máy móc

Quản lý việc giao tiếp cần thiết giữa các máy móc

Dealing with Failures

Lỗi của người làm việc trên bản đồ

Lập bản đồ các nhiệm vụ đã hoàn thành hoặc đang thực hiện tại
Công nhân được đặt lại về trạng thái rảnh và lên lịch lại công việc.

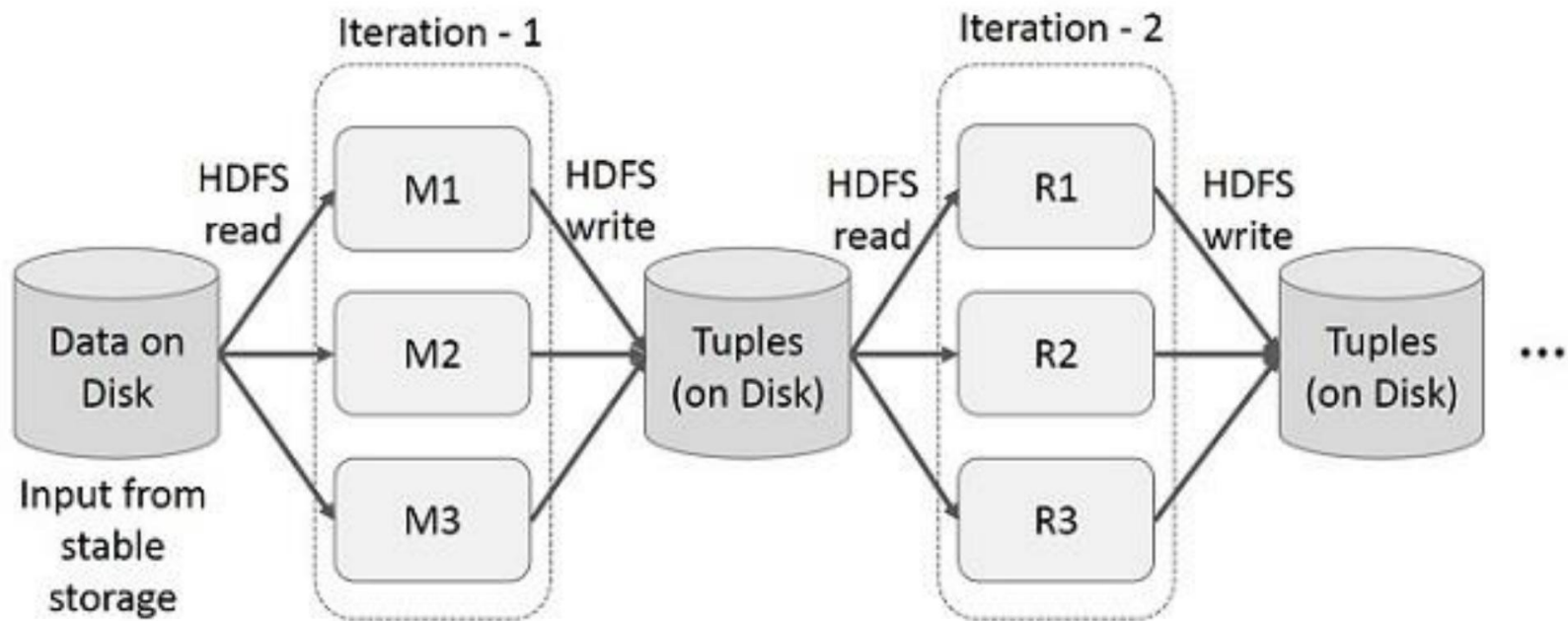
Giảm số lượng nhân viên được thông báo khi nhiệm vụ lập bản đồ hoàn tất.
đã được lên lịch lại cho một nhân viên khác

Giảm thiểu sai sót của người lao động

Chỉ những tác vụ đang thực hiện mới được đặt lại về trạng thái chờ và tác vụ
giảm thiểu được khởi động lại.

Spark: Extends MapReduce

Problems with MapReduce



MapReduce phát sinh chi phí đáng kể do sao chép dữ liệu, thao tác đọc/ghi ổ đĩa và tuần tự hóa.

Kết quả đầu ra của các mapper M được lưu vào đĩa, sắp xếp và sau đó được các reducer R đọc lại (đọc HDFS, ghi HDFS)

Problems with MapReduce

Hai hạn chế chính của MapReduce:

Khó khăn khi lập trình trực tiếp trong MapReduce

Nhiều bài toán/thuật toán dữ liệu lớn không dễ mô tả bằng map-reduce.

Các điểm nghẽn về hiệu năng, hoặc lô xử lý không phù hợp với mục đích sử dụng
các trường hợp

Việc lưu vào ổ đĩa thường chậm hơn nhiều so với việc xử lý dữ liệu trong bộ nhớ.

Tóm lại, MapReduce không phù hợp lắm với các ứng dụng quy mô lớn.

Nhiều khi, người ta cần kết hợp nhiều lệnh map-reduce lại với nhau.
các bước.

Data-Flow Systems

MapReduce sử dụng hai “cấp bậc” của các tác vụ:

Một cái dùng cho Map, cái còn lại dùng cho Reduce.

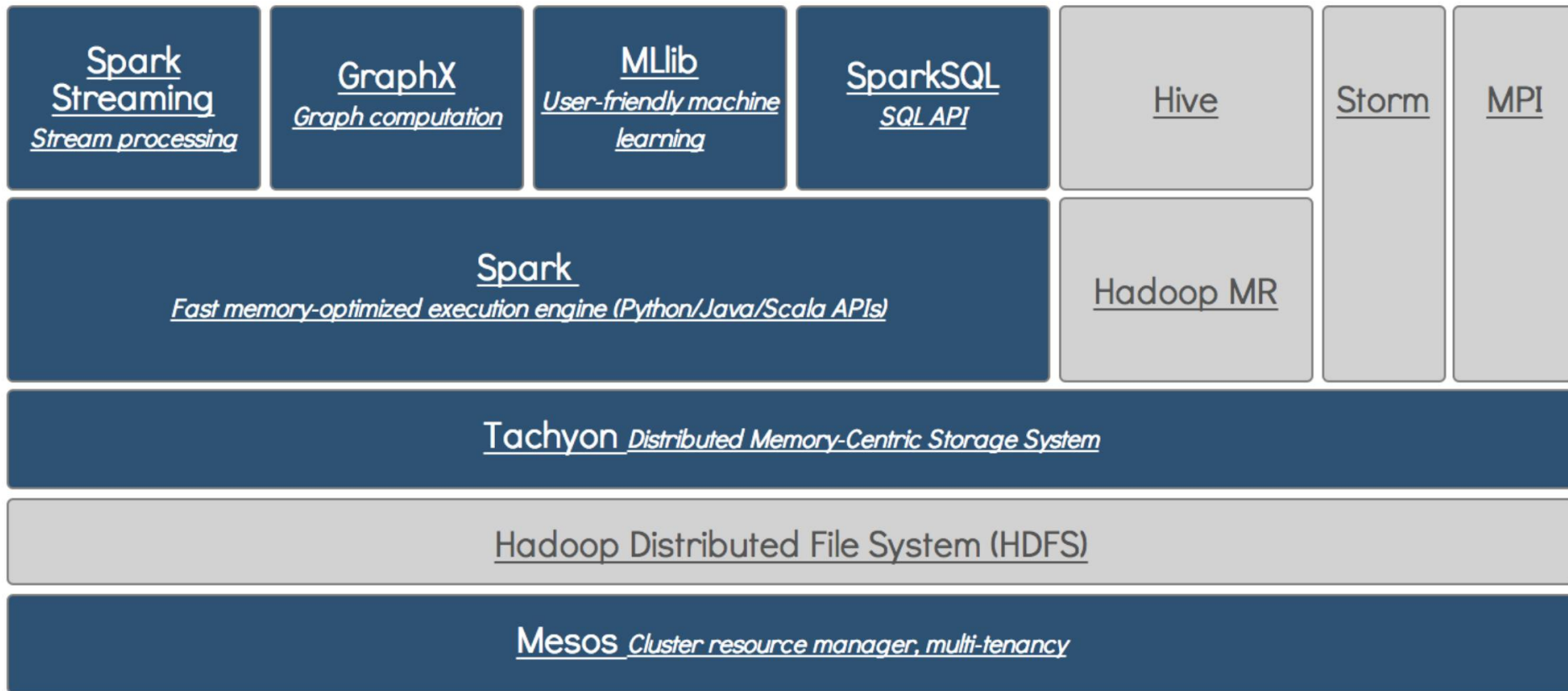
Dữ liệu được truyền từ cấp bậc đầu tiên xuống cấp bậc thứ hai

Hệ thống luồng dữ liệu khái quát hóa điều này theo hai cách:

1. Cho phép số lượng nhiệm vụ/cấp bậc không giới hạn.
2. Cho phép các hàm khác ngoài Map và Reduce.

Nếu luồng dữ liệu chỉ theo một chiều (DAG = có hướng)
(đồ thị không chu trình), chúng ta có thể có thuộc tính chặn và cho phép khôi phục các tác vụ thay vì toàn bộ công việc.

Data Analytics Software Stack



Spark: Most Popular Data-Flow System

Hệ thống tính toán biểu đạt, không giới hạn ở mô hình map-reduce

Bổ sung vào mô hình MapReduce:

Chia sẻ dữ liệu nhanh chóng

Tránh lưu kết quả trung gian vào đĩa

Lưu trữ dữ liệu vào bộ nhớ đệm cho các truy vấn lặp đi lặp lại (ví dụ: cho máy học)

Đồ thị thực thi tổng quát (DAG = đồ thị có hướng không chu trình)
đồ thị)

Chức năng phong phú hơn so với chỉ map và reduce

Tương thích với Hadoop

Spark: Overview

Khái niệm/ý tưởng chính: Tập dữ liệu phân tán có khả năng phục hồi (RDD)

API cấp cao hơn: DataFrames & DataSets

Được giới thiệu trong các phiên bản Spark gần đây hơn

Các API khác nhau dành cho dữ liệu tổng hợp, cho phép tích hợp hỗ trợ SQL.

Spark: RDD

Khái niệm chính: Tập dữ liệu phân tán có khả năng phục hồi (RDD)

Tập hợp các bản ghi được phân vùng

Khái quát hóa các cặp (khóa-giá trị)

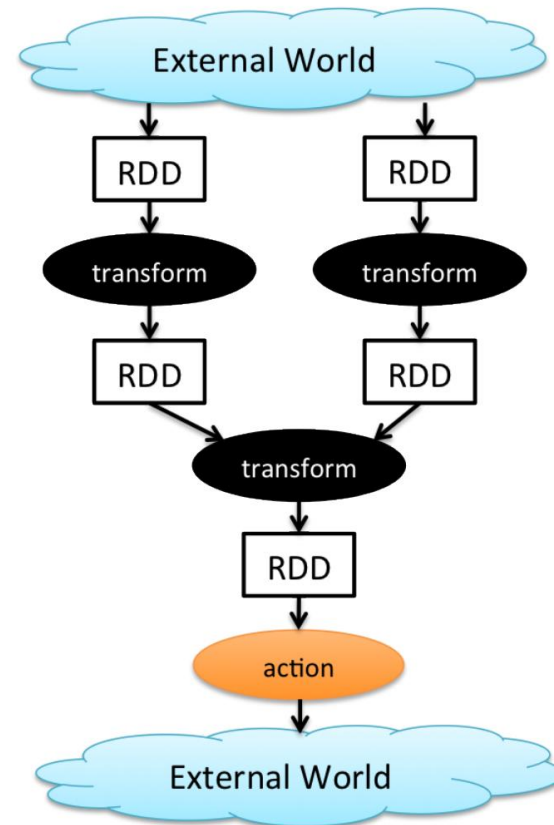
Phân tán khắp cụm, Chỉ đọc

Lưu trữ bộ dữ liệu vào bộ nhớ đệm

Có thể chuyển sang lưu trữ trên ổ đĩa

RDD có thể được tạo từ Hadoop, hoặc bằng cách chuyển đổi các RDD khác (bạn có thể xếp chồng các RDD).

RDD phù hợp nhất cho các ứng dụng áp dụng cùng một thao tác cho tất cả các phần tử của tập dữ liệu.



Spark RDD Operations

Các phép biến đổi xây dựng RDD thông qua các phép toán xác định trên các RDD khác:

Các phép biến đổi bao gồm ánh xạ, lọc, kết hợp, hợp nhất, giao nhau, phân biệt

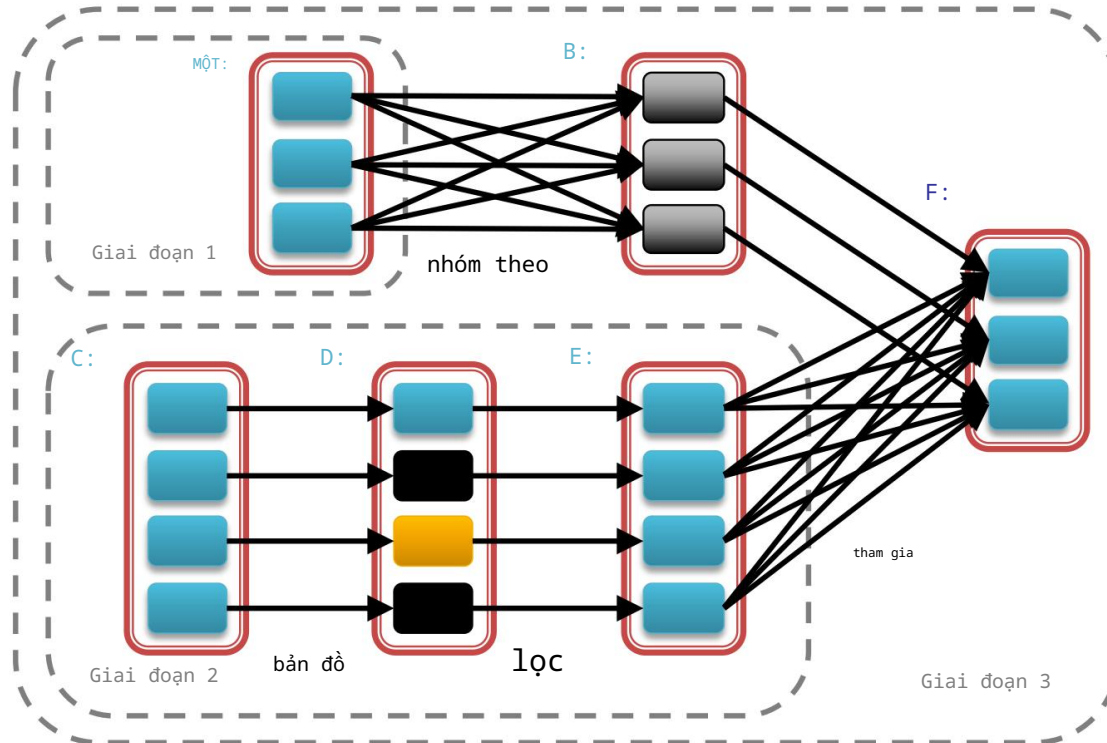
Đánh giá lười biếng: Không có gì được tính toán cho đến khi một hành động yêu cầu.

Các thao tác để trả về giá trị hoặc xuất dữ liệu

Các thao tác bao gồm đếm, thu thập, giảm thiểu, lưu trữ

Các thao tác có thể được áp dụng cho RDD; các thao tác buộc phải thực hiện các phép tính và trả về giá trị.

Task Scheduler: General DAGs



Hỗ trợ đồ thị tác vụ không chu trình có hướng tổng quát

Các chức năng đường ống (pipelines) được tích hợp khi có thể.

Tái sử dụng dữ liệu và tính cục bộ dựa trên bộ nhớ đệm

Có tính đến việc phân vùng để tránh xáo trộn dữ liệu

Higher-Level API: DataFrame & Dataset

DataFrame:

Không giống như RDD, dữ liệu được tổ chức thành các phần tử có tên.

Các cột, ví dụ như một **bảng trong cơ sở dữ liệu quan hệ**.

Áp đặt cấu trúc lên một tập hợp dữ liệu phân tán, cho phép trừu tượng hóa ở cấp độ cao hơn.

Tập dữ liệu:

Mở rộng API của DataFrame, cung cấp giao diện lập trình hướng đối tượng, an toàn kiểu dữ liệu (phát hiện lỗi trong quá trình biên dịch)

Cả hai đều được xây dựng trên công cụ Spark SQL. Cả hai đều có thể được chuyển đổi ngược lại thành RDD.

Useful Libraries for Spark

Spark SQL

xử lý dữ liệu quan hệ có khả năng mở rộng

Spark Streaming

Xử lý luồng dữ liệu trực tiếp

MLlib

học máy có khả năng mở rộng

GraphX

thao tác đồ thị

Mở rộng Spark RDD với trừu tượng hóa đồ thị:
một đa đồ thị có hướng với các thuộc tính được
gắn vào mỗi đỉnh và cạnh.

Spark vs. Hadoop MapReduce

Hiệu năng: Spark thường nhanh hơn nhưng cần lưu ý một số hạn chế.

Spark có thể xử lý dữ liệu trong bộ nhớ; Hadoop MapReduce lưu trữ dữ liệu trở lại ổ đĩa sau khi thực hiện thao tác map hoặc reduce.

Nhìn chung, Spark hoạt động tốt hơn MapReduce, nhưng nó thường cần rất nhiều bộ nhớ để hoạt động hiệu quả; nếu có các dịch vụ khác tiêu tốn nhiều tài nguyên hoặc không thể chứa hết trong bộ nhớ, hiệu suất của Spark sẽ giảm.

MapReduce dễ dàng hoạt động song song với các dịch vụ khác với sự khác biệt nhỏ về hiệu năng, và hoạt động tốt với các tác vụ xử lý một lần (1-pass) mà nó được thiết kế cho.

Dễ sử dụng: Spark dễ lập trình hơn (API cấp cao hơn)

Xử lý dữ liệu: Spark tổng quát hơn

Problems Suited for MapReduce

Example: Host size

Giả sử chúng ta có một kho dữ liệu web lớn.

Xem tệp siêu dữ liệu

Các dòng có dạng: (URL, kích thước, ngày tháng, .)

Đối với mỗi máy chủ, hãy tìm tổng số byte.

Tức là, tổng kích thước trang cho tất cả các URL từ máy chủ cụ thể đó

Các ví dụ khác:

Phân tích liên kết và xử lý đồ thị

Thuật toán học máy

Example: Language Model

Dịch máy thống kê:

Cần đếm số lần xuất hiện của mỗi nhóm 5 từ
Trình tự này xuất hiện trong một lượng lớn tài liệu.

Rất dễ dàng với MapReduce:

Bản đồ:

Trích đoạn (chuỗi 5 từ, đếm) từ tài liệu

Giảm thiểu:

Cộng các số đếm lại

Example: Join By Map-Reduce

Tính phép nối tự nhiên $R(A,B)$ $S(B,C)$

R và S được lưu trữ trong các tệp tin.

Bộ dữ liệu (tuple) là các cặp (a,b) hoặc (b,c)

MỘT	B
a1	b1
a2	b1
a3	b2
a4	b3

R

B	C
b2	c1
b2	c2
b3	c3

S

=

MỘT	C
a3	c1
a3	c2
a4	c3

Map-Reduce Join

Sử dụng hàm băm h từ các giá trị B đến $1 \dots k$

Quy trình lập bản đồ biến đổi như sau:

Mỗi bộ dữ liệu đầu vào $R(a,b)$ được chuyển đổi thành cặp khóa-giá trị $(b,(a,R))$

Mỗi bộ dữ liệu đầu vào $S(b,c)$ thành $(b,(c,S))$

Các tiến trình Map gửi từng cặp khóa-giá trị với khóa b đến tiến trình Reduce $h(b)$

Hadoop thực hiện việc này tự động; chỉ cần cho nó biết k là gì.

Mỗi tiến trình Reduce khớp với tất cả các cặp $(b,(a,R))$ với tất cả $(b,(c,S))$ và đầu ra (a,b,c) .

Problems NOT suitable for MapReduce

MapReduce rất phù hợp cho: Các

bài toán yêu cầu truy cập dữ liệu tuần tự

Các tác vụ xử lý theo lô lớn (không tương tác, thời gian thực)

MapReduce không hiệu quả đối với các bài toán
yêu cầu truy cập dữ liệu ngẫu nhiên (hoặc không đều):

Đồ thị

Dữ liệu phụ thuộc lẫn nhau

Học máy

So sánh nhiều cặp sản phẩm

Cost Measures for Algorithms

Trong MapReduce, chúng ta định lượng chi phí của một thuật toán bằng công thức:

1. Chi phí truyền thông = tổng I/O của tất cả các thao tác quy trình
2. Chi phí truyền thông đã thực hiện = giá trị lớn nhất của I/O dọc theo bất kỳ đường dẫn nào
3. Chi phí tính toán (đã trôi qua) tương tự, nhưng chỉ tính thời gian chạy của các tiến trình.

Lưu ý rằng ở đây ký hiệu Big-O không phải là cách hữu ích nhất (việc bổ sung thêm máy móc luôn là một lựa chọn).

Example: Cost Measures

Đối với thuật toán map-reduce:

Chi phí truyền thông = kích thước tệp đầu vào + $2 \times$ (tổng kích thước của tất cả các tệp được truyền từ các tiến trình Map đến các tiến trình Reduce) + tổng kích thước đầu ra của các tiến trình Reduce.

Hỏi: Tại sao lại có hệ số $2 \times$?

Chi phí truyền thông đã phát sinh là tổng của đầu vào + đầu ra lớn nhất cho bất kỳ quy trình ánh xạ nào, cộng với chi phí tương tự cho bất kỳ quy trình giảm nào.

What Cost Measures Mean

Chi phí đầu vào/đầu ra (giao tiếp) hoặc chi phí xử lý (tính toán) chiếm ưu thế.

Bỏ qua cái này hoặc cái kia

Tổng chi phí cho biết số tiền bạn phải trả tiền thuê dịch vụ điện toán đám mây.

Chi phí phát sinh là thời gian thực tế khi sử dụng song song hóa

Cost of Map-Reduce Join of R and S

Tổng chi phí liên lạc khi kết nối R và S:

$$= O(|R| + |S| + |R| \cdot |S|)$$

Chi phí truyền thông đã thực hiện $= O(s)$

Chúng ta sẽ chọn k và số lượng tiến trình Map sao cho giới hạn I/O s được tuân thủ.

Chúng ta đặt một giới hạn s cho lượng đầu vào hoặc đầu ra mà bất kỳ tiến trình nào có thể có. s có thể là:

Những gì phù hợp với bộ nhớ chính

Những gì có thể lưu trữ trên ổ đĩa cục bộ

Với các chỉ số phù hợp, chi phí tính toán tỷ lệ thuận với kích thước đầu vào + đầu ra.

Vì vậy, chi phí tính toán cũng tương tự như chi phí truyền thông.